

BỘ MÔN KHOA HỌC DỮ LIỆU

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

LAB 03 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT
THS. PHAN HỒ VIỆT TRƯỜNG



HỌC KỲ 3 – NĂM HỌC 2025-2026



KHÓA K30 & K31

#THUẬT TOÁN TÌM KIẾM MÙ

**Tìm kiếm không chính xác (tìm kiếm mù)*

+Nó còn được gọi là chiến lược tìm kiếm mù hoặc kiểm soát mù, được đặt tên như vậy vì chỉ có thông tin về bài toán đặt ra và không có thông tin bổ sung nào khác về các trạng thái. Kỹ thuật tìm kiếm như thế này là tìm kiếm toàn bộ không gian trạng thái để tìm lời giải.

+Tìm kiếm chiều rộng (BFS) & tìm kiếm chiều sâu (DFS) là những ví dụ về tìm kiếm không chính xác.

#THUẬT TOÁN TÌM KIẾM MÙ

**Chiến lược tìm kiếm mù (Uninformed search strategies)*

+Chiến lược tìm kiếm mù chỉ sử dụng thông tin có sẵn được định nghĩa trong bài toán.

+Các thuật toán tìm kiếm mù:

- Tìm kiếm theo chiều rộng (Breadth-first search)
- Tìm kiếm theo chiều sâu (Depth-first search)

#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

```
from collections import defaultdict
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def BFS(self, u):
        visited = [False] * (len(self.graph))

        queue = []

        visited[u] = True
        queue.append(u)

        while queue:
            u = queue.pop(0)
            print(u, end=' ')

            for i in self.graph[u]:
                if not visited[i]:
                    queue.append(i)
                    visited[i] = True
```

```
if __name__ == '__main__':
    g = Graph()
    g.addEdge(0, 1)
    g.addEdge(1, 0)
    g.addEdge(0, 2)
    g.addEdge(2, 0)
    g.addEdge(2, 4)
    g.addEdge(4, 2)
    g.addEdge(1, 4)
    g.addEdge(4, 1)
    g.addEdge(1, 3)
    g.addEdge(3, 1)
    g.addEdge(3, 4)
    g.addEdge(4, 3)
    g.addEdge(3, 5)
    g.addEdge(5, 3)
    g.addEdge(5, 4)
    g.addEdge(4, 5)

    print("BFS - duyệt tìm kiếm chiều rộng bắt đầu từ đỉnh 0")
    g.BFS(0)
```

#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

BFS - duyệt tìm kiếm chiều rộng bắt đầu từ đỉnh 0

0 1 2 4 3 5

#Yêu cầu: mô tả BFS, duyệt tìm kiếm và vẽ đồ thị?

#THỰC HÀNH 02

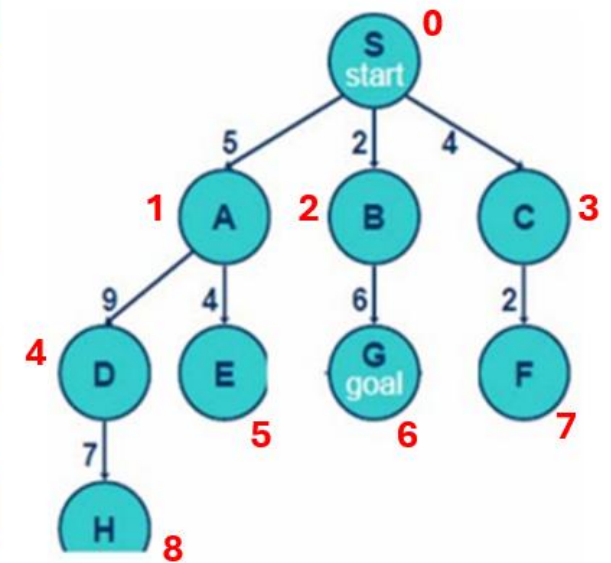
#Tìm kiếm theo chiều rộng

BFS - duyệt tìm kiếm chiều rộng bắt đầu từ đỉnh 0

0 1 2 3 4 5 6 7 8

node	Queue	Father
	S	
S	A, B, C	Father[A,B,C]=S
A	B, C, D, E	Father[D,E]=A
B	C,D,E,G	Father[G]=B
C	D, E, G, F	Father[F]=C
D	E,G, F, H	Father[H]=D
E	G, F, H	
G	F, H	

Giá trị các biến trong
giải thuật theo chiều rộng



Cây tìm kiếm của giải thuật theo chiều rộng

#Yêu cầu: mô tả BFS, duyệt tìm kiếm và vẽ đồ thị?

#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu

```
from collections import defaultdict
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DFSUtil(self, v, visited):
        visited[v] = True
        print(v, end=" ")

        for i in self.graph[v]:
            if visited[i] == False:
                self.DFSUtil(i, visited)

    def DFS(self, v):
        visited = [False] * (max(self.graph) + 1)

        self.DFSUtil(v, visited)
```

```
if __name__ == '__main__':
    g = Graph()
    g.addEdge(0, 1)
    g.addEdge(1, 0)
    g.addEdge(0, 2)
    g.addEdge(2, 0)
    g.addEdge(2, 4)
    g.addEdge(4, 2)
    g.addEdge(1, 4)
    g.addEdge(4, 1)
    g.addEdge(1, 3)
    g.addEdge(3, 1)
    g.addEdge(3, 4)
    g.addEdge(4, 3)
    g.addEdge(3, 5)
    g.addEdge(5, 3)
    g.addEdge(5, 4)
    g.addEdge(4, 5)
    print("DFS - Duyệt tìm kiếm từ vertex 0")
    g.DFS(0)
```

#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu

DFS - Duyệt tìm kiếm từ vertex 0

0 1 4 2 3 5

#Yêu cầu: mô tả DFS, duyệt tìm kiếm và vẽ đồ thị?

#THỰC HÀNH 04

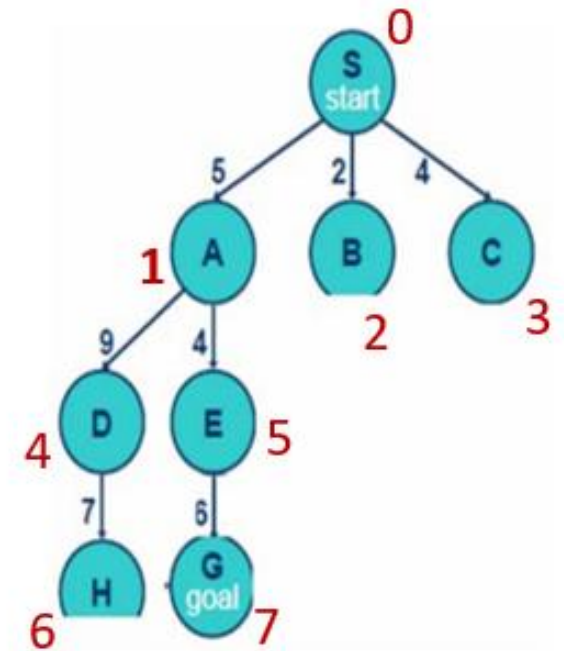
#Tìm kiếm theo chiều sâu

DFS - Duyệt tìm kiếm từ vertex 0

0 1 4 6 5 7 2 3

node	Stack	father
	S	
S	A, B, C	Father[A,B,C]=S
A	D, E, B, C	Father[D,E]=A
D	H, E, B, C	Father[H]=D
H	E, B, C	
E	G, B, C	Father[G]=E
G		

Giá trị các biến trong
giải thuật theo chiều sâu



Cây tìm kiếm của giải thuật theo chiều

#Yêu cầu: mô tả DFS, duyệt tìm kiếm và vẽ đồ thị?

#THỰC HÀNH 05

#Heuristic algorithm - DFS

```
from collections import deque
class Graph:
    def __init__(self, adjac_lis):
        self.adjac_lis = adjac_lis

    def get_neighbors(self, v):
        return self.adjac_lis[v]

    #hàm heuristic có giá trị bằng nhau cho tất cả các nút
    def h(self, n):
        H = {
            'A': 1,
            'B': 1,
            'C': 1,
            'D': 1,
            'E': 1,
            'Z': 1
        }

        return H[n]
```

```
def heuristic_alg(self, start, stop):

    open_lst = set([start])
    closed_lst = set([])

    poo = {}
    poo[start] = 0

    par = {}
    par[start] = start

    while len(open_lst) > 0:
        n = None

        for v in open_lst:
            if n == None or poo[v] + self.h(v) < poo[n] + self.h(n):
                n = v;

        if n == None:
            print('Path does not exist!')
            return None
```

#THỰC HÀNH 05

#Heuristic algorithm - DFS

```
if n == stop:
    reconst_path = []
    while par[n] != n:
        reconst_path.append(n)
        n = par[n]
    reconst_path.append(start)
    reconst_path.reverse()
    print('Path found: {}'.format(reconst_path))
    return reconst_path

for (m, weight) in self.get_neighbors(n):
    if m not in open_lst and m not in closed_lst:
        open_lst.add(m)
        par[m] = n
        poo[m] = poo[n] + weight
    else:
        if poo[m] > poo[n] + weight:
            poo[m] = poo[n] + weight
            par[m] = n
            if m in closed_lst:
                closed_lst.remove(m)
                open_lst.add(m)

    open_lst.remove(n)
    closed_lst.add(n)
    print('Path does not exist!')
    return None
```

#THỰC HÀNH 05

#Heuristic algorithm - DFS

```
if __name__ == '__main__':  
    adjac_lis = {  
        'A': [('B', 4), ('C', 2)],  
        'B': [('A', 4), ('D', 10), ('E', 12)],  
        'C': [('A', 2), ('E', 7)],  
        'D': [('B', 10), ('E', 6), ('Z', 15)],  
        'E': [('B', 12), ('C', 7), ('D', 6), ('Z', 9)],  
        'Z': [('D', 15), ('E', 9)]  
    }  
    g = Graph(adjac_lis)  
    start_node = str(input("Enter the start node:"))  
    stop_node = str(input("Enter the stop node:"))  
    g.heuristic_alg(start_node, stop_node)  
  
#out:
```

Enter the start node:A

Enter the stop node:Z

Path found: ['A', 'C', 'E', 'Z']

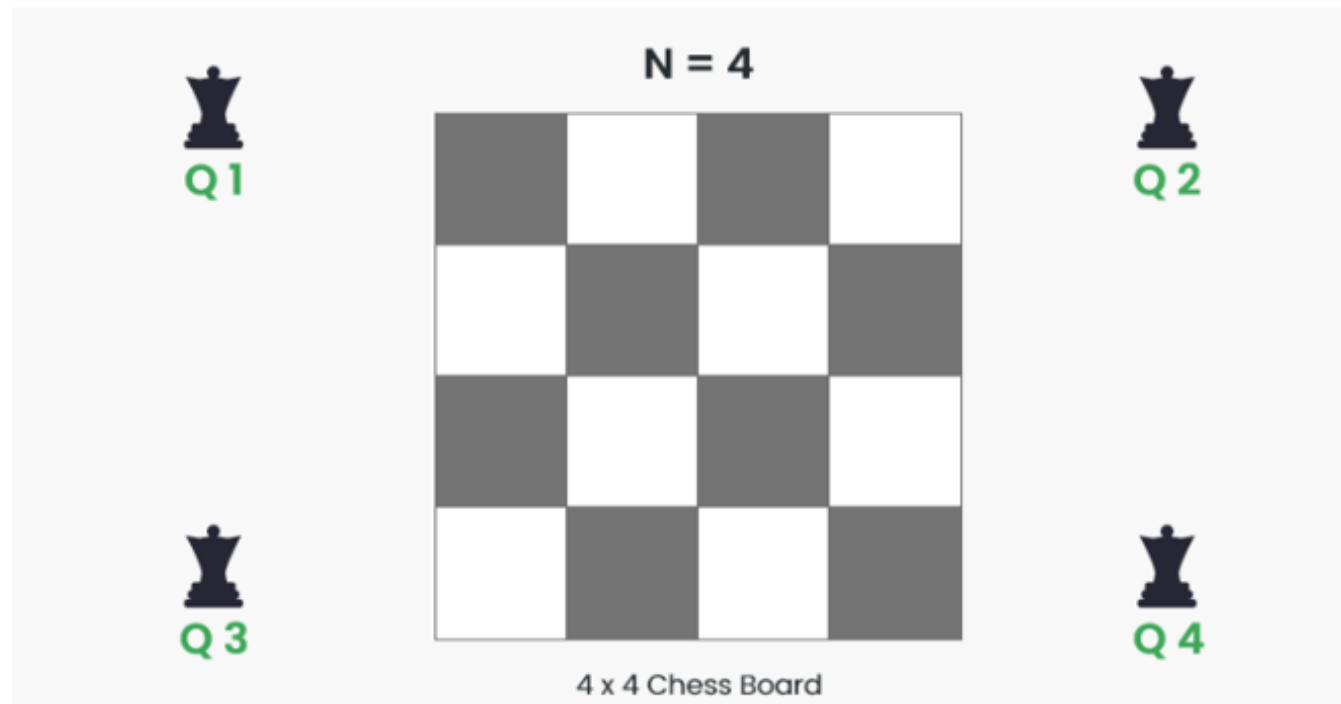
#Yêu cầu: mô tả DFS, duyệt tìm kiếm và vẽ đồ thị?

#THỰC HÀNH 06

#4-Queens State

+Bài toán $n = 4$ Queen trên một bàn cờ vua có 4×4 .

+Đặt ra không có hai Queen nào tấn công lẫn nhau. Ràng buộc điều kiện là không được phép đặt hai Queen trên cùng một hàng hoặc trên cùng một cột hoặc cùng một đường chéo.



#THỰC HÀNH 06

#4-Queens State

Step 1:

- +Đặt quân Queen Q1 vào ô (0,0).
- +‘X’ biểu thị các ô không an toàn tức là chúng đang bị Queen Q1 tấn công.
- +Sau đó di chuyển đến hàng tiếp theo.

	0	1	2	3
0	Q1	x	x	x
1	x	x		
2	x		x	
3	x			x

4 x 4 Chess Board

Step 2:

- +Đặt quân Queen Q2 tiếp theo vào ô (1, 2).
- +Di chuyển đến hàng tiếp theo.

	0	1	2	3
0	Q1	x	x	x
1	x	x	Q2	x
2	x	x	x	x
3	x		x	x

4 x 4 Chess Board

#THỰC HÀNH 06

#4-Queens State

Step 3:

- +Hàng 2 không có ô nào an toàn để đặt Queen Q3
- +Vì vậy, quay lại và loại bỏ Queen Q2 khỏi ô (1, 2)

Step 4:

- +Vẫn còn một ô an toàn ở hàng 1 tức là ô (1, 3).
- +Đặt Queen Q2 vào ô (1, 3).

	0	1	2	3
0	Q1	x	x	x
1	x	x	x	Q2
2	x		x	x
3	x	x		x

4 x 4 Chess Board

#THỰC HÀNH 06

Step 5:

+Đặt Queen Q3 vào ô (2, 1)

	0	1	2	3
0	Q1	x	x	x
1	x	x	x	Q2
2	x	Q3	x	x
3	x	x	x	x

4 x 4 Chess Board

Step 6:

+Không có ô nào để đặt Queen Q4 vào hàng 3.
+Quay lại và loại Queen Q3 khỏi hàng 2.
+Tiếp tục không có ô an toàn nào khác ở hàng 2, vì vậy quay lại và loại Queen Q2 khỏi hàng 1.
+Queen Q1 sẽ bị loại khỏi ô (0,0) và di chuyển đến ô an toàn tiếp theo tức là (0, 1).

Step 7:

+Đặt Queen Q1 vào ô (0, 1) và di chuyển đến hàng tiếp theo.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	
2		x		x
3		x		

4 x 4 Chess Board

#THỰC HÀNH 06

Step 8:

+Đặt Queen Q2 vào ô (1, 3) và di chuyển sang hàng tiếp theo.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2		x	x	x
3		x		x

4 x 4 Chess Board

Step 9:

+Đặt Queen Q3 vào ô (2, 0) và di chuyển sang hàng tiếp theo.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x		x

4 x 4 Chess Board

#THỰC HÀNH 06

Step 10:

+Đặt Queen Q4 vào ô (3, 2) và di chuyển đến hàng tiếp theo.

+Kết luận: đây là giải pháp được chấp nhận.

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x	Q4	x

4 x 4 Chess Board

#THỰC HÀNH 06

#4-Queens State

```
import numpy as np
#Kiểm tra trạng thái hiện tại là giải pháp hợp lệ hay không
def is_valid_state(state, num_queens):
    return len(state) == num_queens

def get_candidates(state, num_queens):
    if not state: return range(num_queens)
    position = len(state)
    candidates = set(range(num_queens))
    for row, col in enumerate(state):
        candidates.discard(col)
        dist = position - row
        candidates.discard(col + dist)
        candidates.discard(col - dist)
    return candidates

#Đệ quy, tìm kiếm theo chiều sâu để tìm ra giải pháp hợp lệ
def search(state, solutions, num_queens):
    if is_valid_state(state, num_queens):
        solutions.append(state.copy())
        print(f"Valid State Found: {state}")

    for candidate in get_candidates(state, num_queens):
        state.append(candidate)
        search(state, solutions, num_queens)
        print(f"backtracking from: {state}")
        state.remove(candidate)
```

#THỰC HÀNH 06

#4-Queens State

```
def solve(num_queens):  
    solutions = []  
    state = []  
    search(state, solutions, num_queens)  
    return solutions  
  
if __name__ == "__main__":  
    num_queens = int(input("Enter number of queens n = "))  
    solutions = solve(num_queens)  
    for solution in solutions:  
        board = np.full((num_queens, num_queens), "-")  
        for row, col in enumerate(solution):  
            board[row][col] = 'Q'  
        print(f'\nSolution: {solution}')  
        print(board)
```

#out:

```
Solution: [1, 3, 0, 2]  
[['-' 'Q' '-' '-']  
['-' '-' '-' 'Q']  
['Q' '-' '-' '-']  
['-' '-' 'Q' '-']]
```

```
Solution: [2, 0, 3, 1]  
[['-' '-' 'Q' '-']  
['Q' '-' '-' '-']  
['-' '-' '-' 'Q']  
['-' 'Q' '-' '-']]
```

#Yêu cầu: mô tả bài toán 4 - Queens & số solution?

#THỰC HÀNH 07

#8-Queens State

```
import numpy as np
#Kiểm tra trạng thái hiện tại là giải pháp hợp lệ hay không
def is_valid_state(state, num_queens):
    return len(state) == num_queens

def get_candidates(state, num_queens):
    if not state: return range(num_queens)
    position = len(state)
    candidates = set(range(num_queens))
    for row, col in enumerate(state):
        candidates.discard(col)
        dist = position - row
        candidates.discard(col + dist)
        candidates.discard(col - dist)
    return candidates

#Đệ quy, tìm kiếm theo chiều sâu để tìm ra giải pháp hợp lệ
def search(state, solutions, num_queens):
    if is_valid_state(state, num_queens):
        solutions.append(state.copy())
        # print(f"Valid State Found: {state}")

    for candidate in get_candidates(state, num_queens):
        state.append(candidate)
        search(state, solutions, num_queens)
        #print(f"backtracking from: {state}")
        state.remove(candidate)
```

#THỰC HÀNH 07

#8-Queens State

```
def solve(num_queens):
    solutions = []
    state = []
    search(state, solutions, num_queens)
    return solutions

if __name__ == "__main__":
    num_queens = int(input("Enter number of queens n = "))
    solutions = solve(num_queens)
    i=0
    for solution in solutions:
        i+=1
        board = np.full((num_queens, num_queens), "-")
        for row, col in enumerate(solution):
            board[row][col] = 'Q'
        print(f'\nSolution: {solution}')
        print(board)
    print('Tổng số Giải pháp là:', len(solutions))
```

#out: . . .

```
Solution: [7, 3, 0, 2, 5, 1, 6, 4]
[['-' '-' '-' '-' '-' '-' '-' 'Q']
 ['- ' '-' '-' 'Q' '-' '-' '-' '-']
 ['Q' '-' '-' '-' '-' '-' '-' '-']
 ['- ' '-' 'Q' '-' '-' '-' '-' '-']
 ['- ' '-' '-' '-' '-' 'Q' '-' '-']
 ['- ' 'Q' '-' '-' '-' '-' '-' '-']
 ['- ' '-' '-' '-' '-' '-' 'Q' '-']
 ['- ' '-' '-' '-' 'Q' '-' '-' '-']]
Tổng số Giải pháp là: 92
```

#Yêu cầu: mô tả bài toán 8 - Queens & số solution?

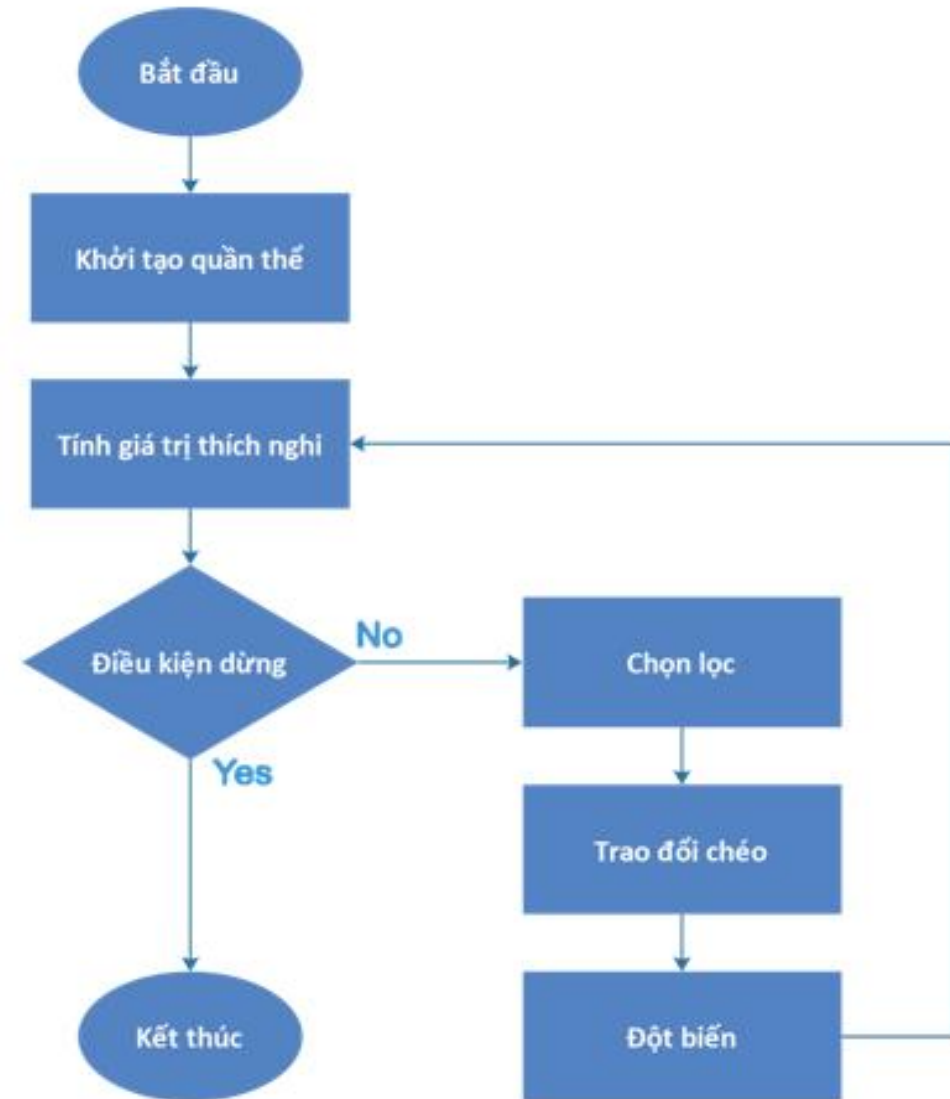
#THỰC HÀNH 08

#thuật toán Di truyền

- **Khởi tạo quần thể:** Sinh ra ngẫu nhiên một quần thể gồm n cá thể (trong đó n là lời giải cho bài toán).
- **Tính giá trị thích nghi:** Ước lượng độ thích nghi của mỗi cá thể.
- **Điều kiện dừng:** Kiểm tra điều kiện để kết thúc giải thuật.
- **Chọn lọc:** Chọn hai cá thể bố mẹ từ quần thể cũ theo độ thích nghi của chúng (cá thể có độ thích nghi càng cao thì càng có nhiều khả năng được chọn).
- **Trao đổi chéo:** Với một xác suất được chọn, trao đổi chéo hai cá thể bố mẹ để tạo ra một cá thể mới.
- **Đột biến:** Với một xác suất đột biến được chọn, biến đổi cá thể mới.
- **Chọn kết quả:** Nếu thỏa mãn điều kiện dừng thì giải thuật kết thúc và chọn được lời giải tốt nhất trong quần thể hiện tại.

#THỰC HÀNH 08

#thuật toán Di truyền



#THỰC HÀNH 08

#Genetic Algorithms – thuật toán di truyền

```
import numpy as np
#định nghĩa hàm fitness_function - hàm số âm
def fitness_func(individual):
    return -np.sum(individual**2)

#tạo quần thể ban đầu
def generate_population(size, dim):
    return np.random.rand(size, dim)

#định nghĩa hàm Genetic algorithm
def genetic_algorithm(population, fitness_func, n_generations=100,
mutation_rate=0.01):
    for _ in range(n_generations):
        population = sorted(population, key=fitness_func, reverse=True)
        next_generation = population[:len(population)//2].copy()
        while len(next_generation) < len(population):
            parents_indices = np.random.choice(len(next_generation), 2,
replace=False)
            parent1, parent2 = next_generation[parents_indices[0]],
next_generation[parents_indices[1]]
            crossover_point = np.random.randint(1, len(parent1))
            child = np.concatenate((parent1[:crossover_point],
parent2[crossover_point:]))
            if np.random.rand() < mutation_rate:
                mutate_point = np.random.randint(len(child))
                child[mutate_point] = np.random.rand()
            next_generation.append(child)
        population = np.array(next_generation)
    return population[0]
```

#THỰC HÀNH 08

```
#tham số
population_size = 10
dimension = 5
n_generations = 50
mutation_rate = 0.05
#khởi tạo quần thể
population = generate_population(population_size, dimension)
#run - thuật toán di truyền
best_individual = genetic_algorithm(population, fitness_func,
n_generations, mutation_rate)
print("Best individual:", best_individual)
print("Best fitness:", -fitness_func(best_individual))
#chuyển đổi sang giá trị dương
```

#out:

```
Best individual: [0.04874523 0.164822    0.0404535
0.1900449  0.167309   ]
Best fitness: 0.09528824076783039
```

#nhận xét: input, output?

#THỰC HÀNH 09

#Giải thuật Di truyền

Ví dụ:

- Tìm max : $f(x) = -x^2 + 15$

<i>Integer</i>	<i>Binary code</i>	<i>Integer</i>	<i>Binary code</i>	<i>Integer</i>	<i>Binary code</i>
1	0 0 0 1	6	0 1 1 0	11	1 0 1 1
2	0 0 1 0	7	0 1 1 1	12	1 1 0 0
3	0 0 1 1	8	1 0 0 0	13	1 1 0 1
4	0 1 0 0	9	1 0 0 1	14	1 1 1 0
5	0 1 0 1	10	1 0 1 0	15	1 1 1 1

<i>Chromosome label</i>	<i>Chromosome string</i>	<i>Decoded integer</i>	<i>Chromosome fitness</i>	<i>Fitness ratio, %</i>
X1	1 1 0 0	12	36	16.5
X2	0 1 0 0	4	44	20.2
X3	0 0 0 1	1	14	6.4
X4	1 1 1 0	14	14	6.4
X5	0 1 1 1	7	56	25.7
X6	1 0 0 1	9	54	24.8
			218	100.0

#THỰC HÀNH 09

#Giải thuật Di truyền

#Bài toán: Guessing Password - Đoán mật khẩu

- **Khởi tạo quần thể:** Sinh ra ngẫu nhiên một đoạn dài 11 ký tự (dài bằng với mục tiêu "Hello World").
- **Tính giá trị thích nghi:** Đếm số ký tự trùng giữa đoạn tạo ra và "Hello World".
- **Điều kiện dừng:** Kiểm tra xem số ký tự trùng bằng 11 hay không.
- **Chọn lọc:** Chọn hai cá thể bố mẹ từ quần thể cũ theo độ thích nghi của chúng (cá thể có độ thích nghi càng cao thì càng có nhiều khả năng được chọn).
- **Trao đổi chéo (Không thực hiện ở bài toán này):** Với một xác suất được chọn, trao đổi chéo hai cá thể bố mẹ để tạo ra một cá thể mới.
- **Đột biến:** Với một xác suất đột biến được chọn, biến đổi cá thể mới.
- **Chọn kết quả:** Khi số ký tự trùng bằng 11 thì dừng giải thuật.

#THỰC HÀNH 09

#Giải thuật Di truyền

#Bài toán: Guessing Password - Đoán mật khẩu

#Đánh giá độ mạnh của mật khẩu dựa trên một số tiêu chí

- Mật khẩu được cộng điểm, khi độ dài hơn 8, 12, 17 và 20 ký tự.
- Kiểm tra xem có chữ hoa, chữ thường, ký tự đặc biệt và chữ số không.

#Chấm điểm các mức

- Chức năng bắt đầu với điểm là 0.
 - Cộng thêm 1 điểm cho mỗi ngưỡng độ dài vượt qua.
 - Cộng thêm 1 điểm cho mỗi loại ký tự được sử dụng (chữ hoa, chữ thường, ký tự đặc biệt, chữ số) hoặc trừ 1 điểm.
 - Dựa trên điểm cuối cùng, phân loại mật khẩu thành "Yếu", "Bình thường", "Rất Tốt" hoặc "Rất Mạnh"
 - Kiểm tra xem mật khẩu có phổ biến hay không. Nếu có, chức năng này sẽ trả về cảnh báo.
- Nếu mật khẩu không phổ biến, chức năng sẽ đánh giá độ mạnh của mật khẩu.
- Nếu mật khẩu yếu (điểm < 4), chức năng sẽ đưa ra các đề xuất cụ thể để cải thiện.

#THỰC HÀNH 09

#Giải thuật Di truyền

```
import string
def check_common_password(password):
    with open('passwords.txt', 'r') as f:
        common = f.read().splitlines()
    if password in common:
        return True
    return False

def password_strength(password):
    score = 0
    length = len(password)
    upper_case = any(c.isupper() for c in password)
    lower_case = any(c.islower() for c in password)
    special = any(c in string.punctuation for c in password)
    digits = any(c.isdigit() for c in password)

    characters = [upper_case, lower_case, special, digits]
    if length > 8:
        score += 1
    if length > 12:
        score += 1
    if length > 17:
        score += 1

    if length > 20:
        score += 1
```


#THỰC HÀNH 09

#Giải thuật Di truyền

```
score += sum(characters) - 1

if score < 4:
    return "Weak", score
elif score == 4:
    return "Okay", score
elif 4 < score < 6:
    return "Good", score
else:
    return "Strong", score
def feedback(password):
    if check_common_password(password):
        return "Password was found in a common list. Score: 0/7"

    strength, score = password_strength(password)
    feedback = f"Password strength: {strength} (Score: {score}/7)\n"
    if score < 4:
        feedback += "Suggestions to improve your password:\n"
        if len(password) <= 8:
            feedback += "- Make your password longer (more than 8 characters). \n"
        if not any(c.isupper() for c in password):
            feedback += "- Include uppercase letters.\n"
        if not any(c.islower() for c in password):
            feedback += "- Include lowercase letters.\n"
        if not any(c in string.punctuation for c in password):
            feedback += "- Add special characters (e.g., @, #, $).\n"
        if not any(c.isdigit() for c in password):
            feedback += "- Add numbers.\n"
    return feedback
password = input("Enter the password: ")
print(feedback(password))
```

#THỰC HÀNH 09

#Giải thuật Di truyền

#out:

```
Enter the password: Winter2025@  
Password strength: Okay (Score: 4/7)
```

#nhận xét: input, output – password: 8; 12; 18 ký tự ?

#THỰC HÀNH 10

#thuật toán tiến hóa vi phân đột biến-Differential Evolution (DE)

- Thuật toán tiến hóa khác là thuật toán tiến hóa vi phân nhằm tìm kiếm sự tối ưu của một vấn đề nào đó bằng cách sử dụng các cải tiến cho một giải pháp ứng cử viên.
- Tiến hoá vi phân hoạt động bằng cách đưa ra các giải pháp ứng cử viên mới từ quần thể thông qua một hoạt động được gọi là phép cộng vector (phép đột biến và lai ghép).
- DE thường được thực hiện bằng các hoạt động đột biến và lai ghép để tạo ra các vector mới và thay thế các cá thể có mức độ phù hợp thấp trong quần thể.

#THỰC HÀNH 10

```
import numpy as np
def differential_evolution(objective_func, bounds, pop_size=50,
max_generations=100, F=0.5, CR=0.7, seed=None):
    np.random.seed(seed)
    n_params = len(bounds)
    population = np.random.uniform(bounds[:, 0], bounds[:, 1],
size=(pop_size, n_params))
    best_solution = None
    best_fitness = np.inf
    for generation in range(max_generations):
        for i in range(pop_size):
            target_vector = population[i]
            indices = [idx for idx in range(pop_size) if idx != i]
            a, b, c = population[np.random.choice(indices, 3,
replace=False)]
            mutant_vector = np.clip(a + F * (b - c), bounds[:, 0],
bounds[:, 1])
            crossover_mask = np.random.rand(n_params) < CR
            trial_vector = np.where(crossover_mask, mutant_vector,
target_vector)
            trial_fitness = objective_func(trial_vector)
            if trial_fitness < best_fitness:
                best_fitness = trial_fitness
                best_solution = trial_vector
            if trial_fitness <= objective_func(target_vector):
                population[i] = trial_vector

    return best_solution, best_fitness
```

#THỰC HÀNH 10

```
#hàm mục tiêu (hàm min)
def sphere_function(x):
    return np.sum(x**2)
#xác định tham số
bounds = np.array([[ -5.12, 5.12]] * 10) # Example: 10 parameters in [ -
5.12, 5.12] range
#run tiến hoá DE
best_solution, best_fitness = differential_evolution(sphere_function,
bounds)
#kết quả giải pháp tốt nhất & chất lượng tốt nhất
print("Best solution:", best_solution)
print("Best fitness:", best_fitness)
```

#out:

```
Best solution: [ 0.00159031 -0.00567955 -0.00020499 -0.00573414  0.00164174 -0.0051367
 0.00190499 -0.01349925 -0.00127274 -0.00299844]
Best fitness: 0.00029325913422526543
```

#nhận xét: input, output?

#THỰC HÀNH 11

```
#Bài toán Travelling SalesMan(bằng thuật toán di truyền)
import random
import copy
import math
import matplotlib.pyplot as plt

POPULATION_SIZE = 20    #số quần thể
CITIES_SIZE = 20        #số thành phố
TOUR_SIZE = 21          #số tour du lịch
NUM_EXECUTION = 9999    #số lần thực hiện
population = []
x = []
y = []
tour = [[0 for x in range(TOUR_SIZE)] for y in range(TOUR_SIZE)]
dCidade = [[0 for x in range(POPULATION_SIZE)] for y in
range(POPULATION_SIZE)]
distances = [0 for x in range(POPULATION_SIZE)]
parentsOne = None
parentsTwo = None
costByExecution = []
```

#THỰC HÀNH 11

```
def generateFirstPopulation():    #tạo quần thể đầu tiên
    #tạo đường đi mới
    for _ in range(1, POPULATION_SIZE + 1):
        generatePossiblePath()

def generatePossiblePath():    #tạo mới tập hợp đường đi
    path = []
    for _ in range(1, CITIES_SIZE + 1):
        #tạo mới số cá thể từ 1-20
        randomNum = random.randint(1, 20)
        while (numberExistsInPath(path, randomNum)):
            randomNum = random.randint(1, 20)
        path.append(randomNum)
    population.append(path)

def numberExistsInPath(path, number):
    for i in path:
        if i == number:
            return True
    return False
```

#THỰC HÀNH 11

#Tạo mảng X và mảng Y - khoảng cách trong trục x và y để tính ma trận
nhận dạng trong hàm thích nghi

```
def generateXandY():  
    for _ in range(CITIES_SIZE):  
        randomNumber = random.random()  
        randomNumber = round(randomNumber, 2)  
        x.append(randomNumber)  
  
        randomNumber = random.random()  
        randomNumber = round(randomNumber, 2)  
        y.append(randomNumber)
```

#hoán đổi giữa 2 thành phố trên đường đi, với 5% cơ hội đột biến

```
def mutate(matrix):  
    for i in range(0, len(matrix)):  
        for _ in range(0, len(matrix[i])):  
            ranNum = random.randint(1, 100)  
            if ranNum >= 1 and ranNum <= 5:  
                indexOne = random.randint(0, 19)  
                indexTwo = random.randint(0, 19)  
                auxOne = matrix[i][indexOne]  
                auxTwo = matrix[i][indexTwo]  
                matrix[i][indexOne] = auxTwo  
                matrix[i][indexTwo] = auxOne
```

#THỰC HÀNH 11

#Tạo ma trận Tour, ma trận giống như quần thể, với cột đầu tiên được sao chép ở cuối cột đó, sau cùng, người du lịch luôn phải đến cùng một nơi mà anh ta đã xuất phát

```
def generateTour():
    global tour
    tour = copy.deepcopy(population)
    for ways in tour:
        first = ways[0]
        ways.append(first)

#Tạo mảng có tổng đường đi trong mảng quần thể dựa trên ma trận tour;
def calculateDistances():
    global distances
    distances = [0 for x in range(POPULATION_SIZE)]
    for i in range(len(population)):
        for j in range(len(population[i])):
            firstPos = 19 if tour[i][j] == 20 else tour[i][j]
            secondPos = 19 if tour[i][j+1] == 20 else tour[i][j+1]
            distances[i] += round(dCidade[firstPos][secondPos], 4)
    dict_dist = {i: distances[i] for i in range(0, len(distances))}
    distances = copy.deepcopy(dict_dist)
    return sorted(distances.items(), key=lambda kv: kv[1])
```

#THỰC HÀNH 11

#Tạo ma trận nhận dạng (dCidade) dựa trên mảng x và y, rồi gọi hàm CalculateDistances() để tạo mảng có tổng đường đi tới người dừng sau, theo chu trình;

```
def fitnessFunction():  
    for i in range(len(population)):  
        for j in range(len(population)):  
            dCidade[i][j] = round(math.sqrt(((x[i] - x[j])**2) + ((y[i] - y[j])**2))), 4)  
    return calculateDistances()
```

#hàm roulette, tạo ra hai mảng, mỗi mảng sẽ được sử dụng sau, thực hiện theo chu trình;

```
def rouletteFunction(sorted_x):  
    global parentsOne  
    global parentsTwo  
    arr = []  
    rouletteArr = []  
    for i in range(10):  
        arr.append(sorted_x[i][0])  
    for j in range(len(arr)):  
        for _ in range(10 - j):  
            rouletteArr.append(arr[j])  
    parentsOne = createParents(rouletteArr)  
    parentsTwo = createParents(rouletteArr)
```


#THỰC HÀNH 11

```
#sử dụng hàm rouletteFunction() để tạo ra hai mảng cha;
def createParents(rouletteArr):
    parentArr = []
    for _ in range(5):
        parentArr.append(rouletteArr[random.randint(0, 54)])
    return parentArr

#sử dụng phương pháp tuần hoàn để xem xét thành phố nào bị trùng lặp
hay không
def hasDuplicity(auxArray, usedIndexes):
    for i in range(len(auxArray)):
        for j in range(i, len(auxArray)):
            if i != j and auxArray[i] == auxArray[j]:
                if i in usedIndexes:
                    return j
                else:
                    return i
    return -1
```

#THỰC HÀNH 11

#1. hai phần tử con trong mảng con, thực hiện hoán đổi ngẫu nhiên cho đến khi không còn phần tử trùng lặp

#2. Thay đổi children được tạo ra

#3. Thêm các phần tử children vào mảng quần thể

```
def doCycle(sorted_x):  
    global population  
    children = []  
    for i in range(5):  
        parentOneAux = parentsOne[i]  
        parentTwoAux = parentsTwo[i]  
        usedIndexes = []  
        randomIndexInsideCromossomus = random.randint(0, POPULATION_SIZE -  
1)  
  
        usedIndexes.append(randomIndexInsideCromossomus)  
        childOne = copy.deepcopy(population[parentOneAux])  
        childTwo = copy.deepcopy(population[parentTwoAux])  
  
        valAuxOne = childOne[randomIndexInsideCromossomus]  
        valAuxTwo = childTwo[randomIndexInsideCromossomus]
```

#THỰC HÀNH 11

```
childOne[randomIndexInsideCromossomus] = valAuxTwo
childTwo[randomIndexInsideCromossomus] = valAuxOne

while (hasDuplicity(childOne, usedIndexes) != -1):
    newIndex = hasDuplicity(childOne, usedIndexes)
    usedIndexes.append(newIndex)
    valAuxOne = childOne[newIndex]
    valAuxTwo = childTwo[newIndex]
    childOne[newIndex] = valAuxTwo
    childTwo[newIndex] = valAuxOne

# Sau khi tạo ra phần tử con, thêm chúng vào mảng phần tử con;
children.append(childOne)
children.append(childTwo)

#thay đổi mảng con
mutate(children)

#Tạo bản sao quần thể trước khi thay đổi nó
tempPop = copy.deepcopy(population)

for i in range(10):
    population[i] = copy.deepcopy(tempPop[sorted_x[i][0]])

#điều chỉnh quần thể
for j in range(10, POPULATION_SIZE):
    population[j] = copy.deepcopy(children[j - 10])
```

#THỰC HÀNH 11

```
def main():
    #tạo quần thể, x và y và ma trận tour;
    generateFirstPopulation()
    generateXandY()
    generateTour()

    #vòng lặp từ 0 - 9999 ;
    for _ in range(NUM_EXECUTION):
        sorted_x = fitnessFunction()
        rouletteFunction(sorted_x)
        doCycle(sorted_x)
        generateTour() #lặp ma trận Tour khi quần thể được cập nhật
        costByExecution.append(sorted_x[0][1])
            #Thêm chi phí vào mảng chi phí

    #Tạo giá trị thích hợp cho quần thể cuối cùng
    sorted_x = fitnessFunction()

    print('Total Population: %s' % (POPULATION_SIZE))
    print('Mutation Probability: 5%') #xác suất đột biến
    print('No of Cities: %s' % (CITIES_SIZE))
    print('optimal Path Cost: %s' % sorted_x[0][1])
    print('Best Route: %s' % population[0]) #tuyến đường đi tốt nhất
```

#THỰC HÀNH 11

```
#biểu đồ đường đi
plt.plot(tour[0])
plt.plot(tour[0], 'rs')
plt.axis([0, 20, 0, 20])
plt.xlabel('No of cities') #số thành phố (số điểm)
plt.ylabel("Total population") #tổng số quần thể
plt.title("The route of Salesman")
plt.show()

#biểu đồ chi phí
plt.plot(costByExecution)
plt.xlabel('No of Iterations') #số lần lặp
plt.ylabel("Cost function value (fitness value)") #hàm chi phí
plt.show()

if __name__ == "__main__":
    main()
```

#THỰC HÀNH 11

#out:

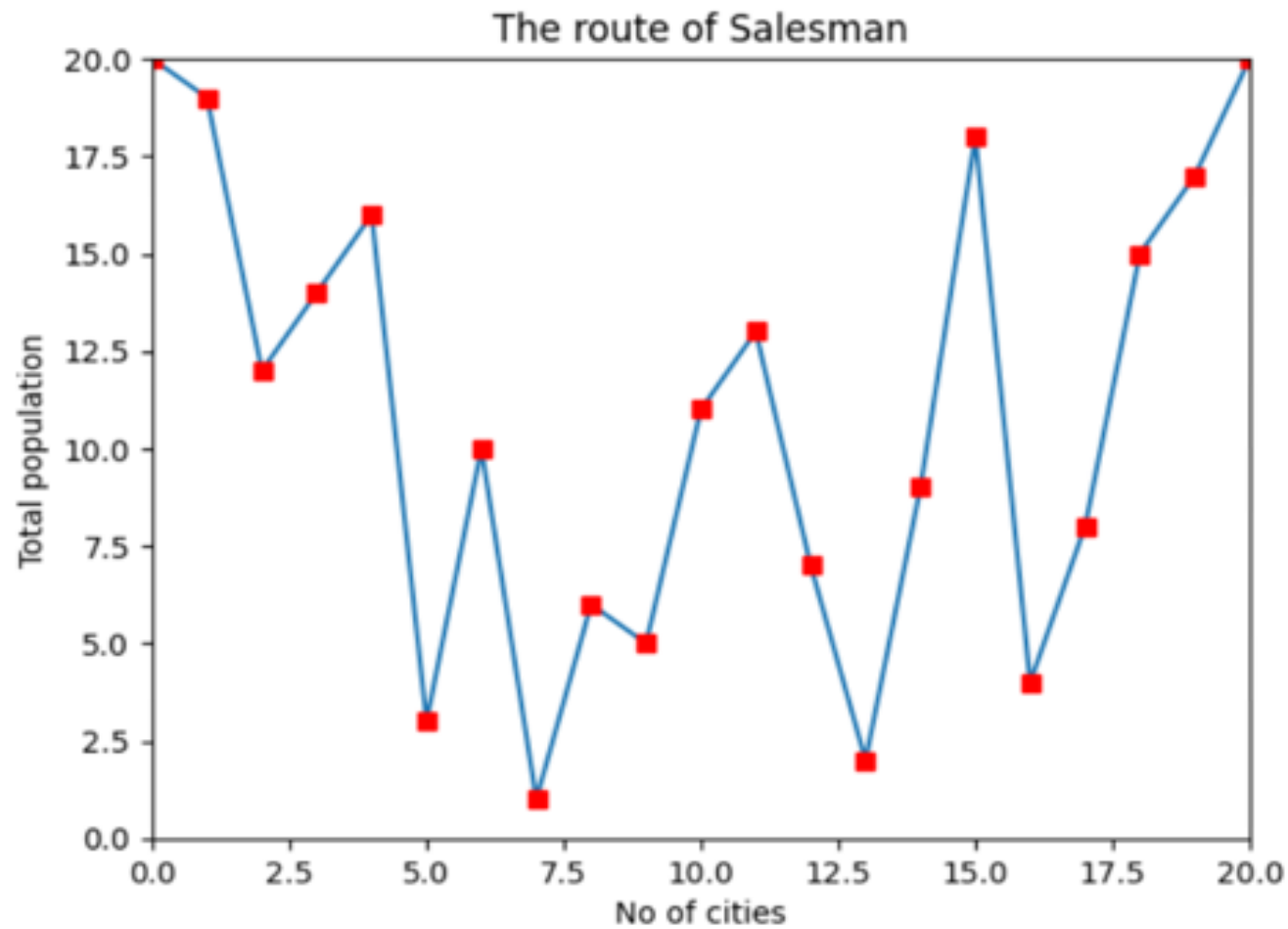
Total Population: 20

Mutation Probability: 5%

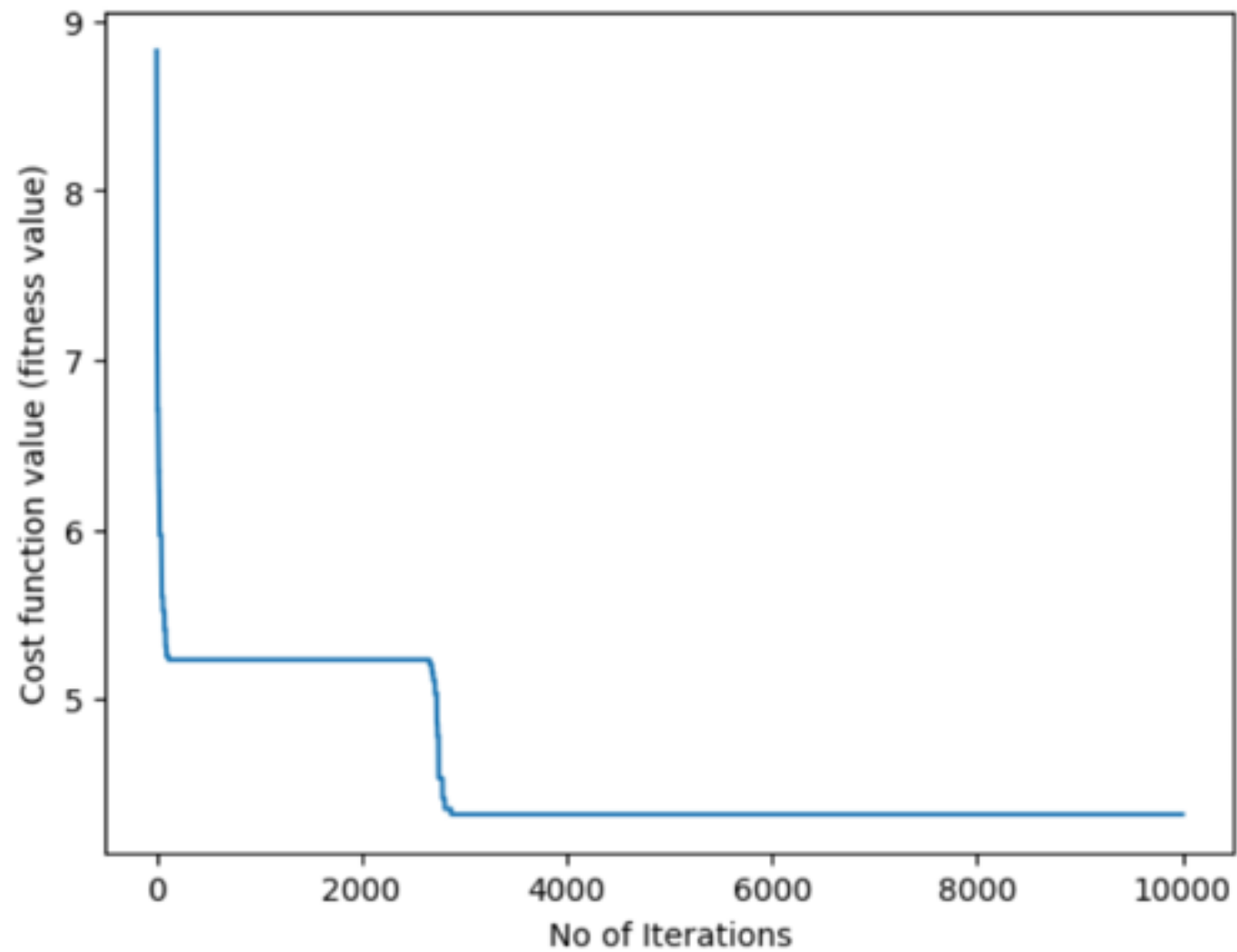
No of Cities: 20

optimal Path Cost: 4.316999999999999

Best Route: [20, 19, 12, 14, 16, 3, 10, 1, 6, 5, 11, 13, 7, 2, 9, 18, 4, 8, 15, 17]



#THỰC HÀNH 11



#nhận xét: input, output & số lần lặp?



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

